Leetcode 875. Koko Eating Bananas

October 14th 2026

Understanding the Problem

We are trying to find the minimum per-hour eating speed (k) that Koko should eat all the banana piles at so that she can finish all the bananas within h hours. Let's illustrate the problem with an example first. Let's assume that we were given these inputs:

$$\texttt{piles} = [3, 6, 7, 11]$$

$$h = 8$$

Notice that if $k \geq \max(\texttt{piles}[i])$, then $h = \texttt{len(piles)}$. In this case, if $k \geq 11$, then $h = 4$. Why? Because Koko can only finish one pile at a time, she will take the same time (1 hr) to finish each pile. There are 4 elements in the array, so $h = 4$. This also means that the minimum value of h has to be the length of the `piles` array.

$$1 \leq \texttt{len(piles)} \leq h$$

From this, how can we determine the value of k? What range of numbers make sense for k to be? Well, $k \geq 1$ since Koko will eat bananas. Recall from above that if $k \geq \max(\texttt{piles})$, then $h = \texttt{len(piles)}$. Therefore the search space for possible values of k is as follows:

$$k = 1, ..., \max(\texttt{piles})$$

Therefore, we have to test the eating rate for each of these numbers to see which rate matches the given value of h.

Solution

Brute Force

We can try each value of k in the search range $1, ..., \max(\texttt{piles})$ and see how long it takes to eat all the bananas for each speed k. Doing this for the example in the introduction, we get:

$$\texttt{piles} = [3, 6, 7, 11], \qquad h = 8$$

$$T(k) = \sum_{p \in \texttt{piles}} \left\lceil \frac{p}{k} \right\rceil$$

k	**Hours per Pile**	$T(k)$	**Feasible?**
1	$[3, 6, 7, 11]$	27	No
2	$[2, 3, 4, 6]$	15	No
3	$[1, 2, 3, 4]$	10	No
4	$[\mathbf{1}, \mathbf{2}, \mathbf{2}, \mathbf{3}]$	**8**	**Yes**
5	$[1, 2, 2, 3]$	8	Yes
6	$[1, 1, 2, 2]$	6	Yes
7	$[1, 1, 1, 2]$	5	Yes
8	$[1, 1, 1, 2]$	5	Yes
9	$[1, 1, 1, 2]$	5	Yes
10	$[1, 1, 1, 2]$	5	Yes
11	$[1, 1, 1, 1]$	4	Yes

Let's examine the time and space complexity of this algorithm:

- Time complexity: $\mathcal{O}(nk)$
  - We have to try each of the k possible rates n times for each element.
- Space complexity: $\mathcal{O}(1)$

As we can see here, the answer to the problem is $k = 4$ since it's the **smallest** value of k that allows Koko to eat all the bananas in 8 hours. Also, notice how the search space is always going to be sorted in increasing order since we are looking through all possible rates starting with 1.
Now, what algorithm could be used to increase the efficiency of search in a sorted search space? The answer is Binary Search!

Binary search optimization

The idea here is to use binary search to reduce the expensive search cost when we are trying possible values of k. In the case above, instead of linear search, we can look at $k = 6$ first and see that it results in Koko eating the bananas in 6 hours. We can slow down the eating speed now, so we can divide the search space into $k = 1, ..., 5$. Repeat the search to find the minimum eating speed, $k = 4$. Note that $k = 5$ also results in $h = 8$, but we need to find the minimum, so we need to search before $k = 5$, even though $k = 5$ gives us the desired result.

If h doesn't exist

If, for example, we were given $h = 9$ instead of $h = 8$, then what would we return? We would return $k = 4$ since it's the smallest value of k that allows Koko to finish all the bananas in $h = 9$ hours. For the binary search code, use `l <= r` and `return l` since we are trying to find the **smallest** acceptable answer.

Time and Space Complexity

- Time complexity: $\mathcal{O}(n \log k)$
- Space complexity: $\mathcal{O}(1)$

Our search space for binary search will be $k = \max(\texttt{piles})$, and we have to test each of the potential k values on all the elements of the array. We aren't using any additional data structures, so our space complexity remains $\mathcal{O}(1)$.